

E-Commerce API

Capstone Project



Fai Alharthi

INTRODUCTION

What This Project Is

API of a small e-commerce website. Two actors interact with it: a User —> who can be a Customer or an Admin and a Merchant, a separate class that stocks the products for sale.

THE ACTORS



Customer

A User role that shops: buys, returns, and sends/receives gift-card balance.



Admin

A User role locked out of shopping — balance fixed at 0, no purchases.



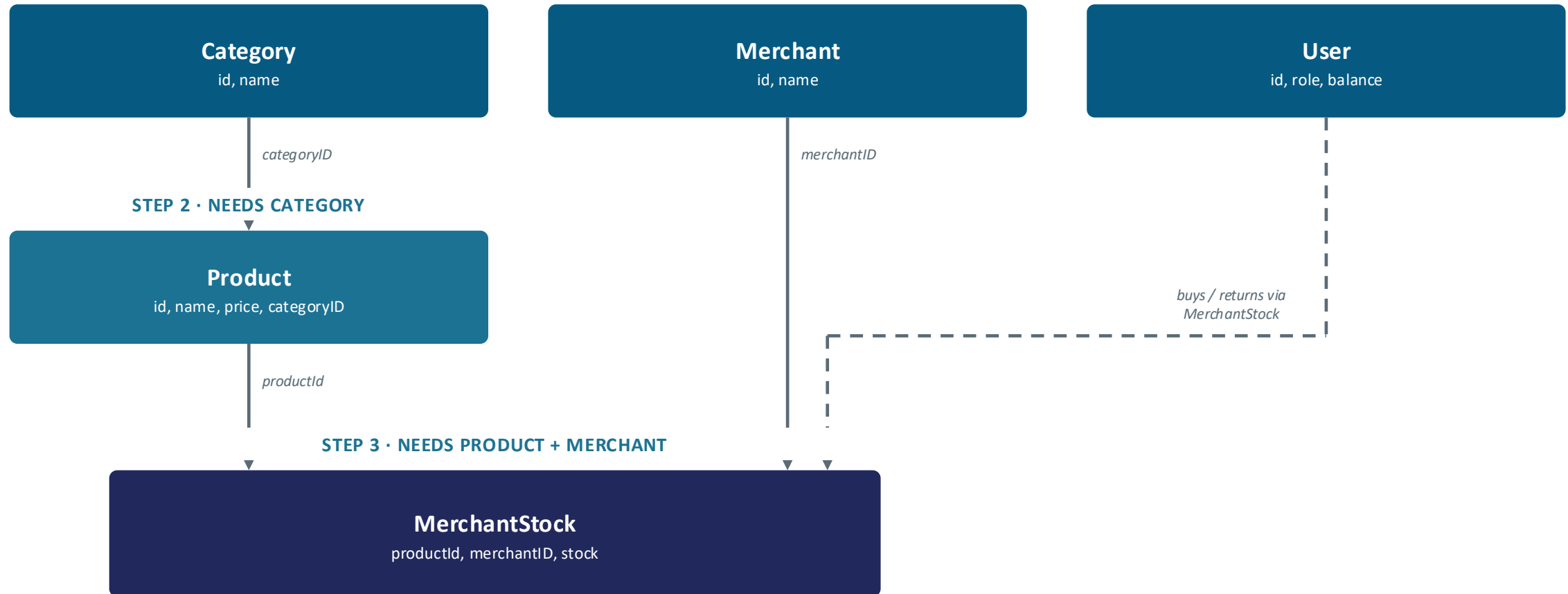
Merchant

A separate class, not a User role — stocks products via MerchantStock.

DATA MODEL

What Must Be Created Before What

STEP 1 • NO DEPENDENCIES — CREATE FIRST, ANY ORDER



Category, Merchant, and User have no dependencies and can be created in any order. Product needs an existing Category. MerchantStock needs both an existing Product and Merchant — only then can a User buy or return through it (dashed line).

Every Request Passes Two Checks



Field validation always runs first — an invalid stock quantity returns its message even when the merchant or product ID is also wrong.

MINIMUM REQUIREMENTS

20 CRUD Endpoints + Requirements #11 & #12

Category

/api/category/v1

- GET /getAllCats
- POST /addCategory
- PUT /updateCat/{id}
- DELETE /deleteCat/{id}

VALIDATIONS

- id, name not empty
- name ≥ 4 chars

Merchant

/api/merchant/v1

- GET /getMerchants
- POST /addMerchant
- PUT /updateMerchant/{id}
- DELETE /deleteMerchant/{id}

VALIDATIONS

- id, name not empty
- name ≥ 4 chars

Product

/api/product/v1

- GET /getProducts
- POST /addProducts
- PUT /updateProduct/{id}
- DELETE /deleteProduct/{id}

VALIDATIONS

- id, name, price, categoryID not empty
- name ≥ 4 chars

MerchantStock

/api/merchantStock/v1

- GET /getStocks
- POST /addStock
- PUT /updateEntireStock/{id}
- DELETE /deleteStock/{id}

VALIDATIONS

- id, productId, merchantID not empty
- stock not empty, ≥ 10

User

/api/user/v1

- GET /getUsers
- POST /addUser
- PUT /updateUser/{id}
- DELETE /deleteUser/{id}

VALIDATIONS

- all fields not empty
- username & password ≥ 6
- valid email format
- role: customer/admin
- balance ≥ 0 (0 for admin)

11

Requirement #11 · Merchant Restocking

updateStock(productId, merchantID, stock)

12

Requirement #12 · Direct Purchase

buyProduct(userID, productId, merchantID)

Endpoints

Beyond the 20 CRUD endpoints, eight more round out the API, two of them (gold) also fulfill Requirements #11 & #12.
the rest are five designed , plus one extra-credit endpoint.

● = *fulfills a Minimum Requirement (#11 / #12)*

PRODUCT & STOCK



productsInRange



applyDiscount



updateStock #11

PURCHASE LIFECYCLE



buyProduct #12



returnProduct



giveBalance

ORDERS & ADMIN



PreviousOrders



displayCustomer

Product & Stock: Beyond Basic CRUD



1 / 5

`productsInRange(categoryID, min, max)`

Filters one category by a price window;
400 if min > max, or nothing matches
in that range.



2 / 5

`applyDiscount(merchantID, productID,
discountPercentage)`

Applies a % off to a merchant's product;
400 if under 5%, or the merchant
doesn't sell it.

AFFECTS OTHER ENDPOINTS

Lowers the product's price for everyone

Each of these layers business rules on top of simple existence checks , the 400 response always names the exact rule that failed.

The Purchase Lifecycle

buyProduct and returnProduct are independent actions any Customer-role User can take.

#12

buyProduct

Checks stock & balance,
charges the customer;
400 if merchant is out of stock

AFFECTS

-1 stock, -balance,
logged for PreviousOrders

3 / 5

returnProduct

Reverses a real purchase;
400 if never purchased,
or already returned

AFFECTS

+1 stock, +balance refunded

4 / 5

Recipient ≠ admin

giveBalance

Sends gift-card balance
between users;
400 if sender's balance is short

AFFECTS

-sender balance,
+recipient balance

Each rule is enforced in the service layer, not just the database , the 400 response always names the exact rule that failed.

Order History & Admin Reporting



PreviousOrders/{userID}

Self-Designed Endpoint

5 / 5

Returns everything a customer has bought.

400 if the caller doesn't exist, or the purchase history is empty.

Read-only, reflects the purchase log that buyProduct writes to.



displayCustomer/{userID}

Extra Credit Endpoint

Admins only

Lists every customer account.

400 for any caller who isn't an admin, same message either way.

Read-only , no effect on other endpoints.